Seasonal Production Planning Calendar

Internship Reference Document

Company	Sharadha Stores
Solution	Seasonal Production Planning Calendar
What it solves	Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity
Why useful	Prevents the most common failure mode of homemade food businesses — running out of stock during peak demand
Duration	01 June 2026 - 30 June 2026 (26 Working Days)
Team size	3 Students per team
Roles	Student 1 - Frontend | Student 2 - Backend | Student 3 - Testing & Deployment

Introduction Session

This section covers everything the instructor should explain in the Introduction Video (Day 1). Students must watch and understand this before any coding begins.

1. About the Company

Sharadha Stores is a home-based food business producing and selling traditional South Indian items — podis, pickles, appalam, ghee, and snacks — through WhatsApp ordering, online platforms, and doorstep delivery. The business has a loyal customer base that values authentic home-style preparation using traditional recipes.

2. The Problem They Face

Sharadha Stores currently handles "admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity" without a dedicated digital system. Prevents the most common failure mode of homemade food businesses — running out of stock during peak demand — but this potential is being lost every day without the right tool.

Problem: Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity. At present, this type of work can become slow or scattered when it depends only on calls, WhatsApp messages, spreadsheets, or manual follow-up.

This results in:

- Without a digital system, admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity is done manually — creating errors, delays, and inconsistency at Sharadha Stores
- Records are scattered across spreadsheets, WhatsApp messages, and paper registers with no centralised tracking
- Management cannot generate real-time reports or visibility without manually compiling data from multiple sources
- The absence of a structured digital workflow means time is wasted on repetitive manual tasks every day
- Prevents the most common failure mode of homemade food businesses — running out of stock during peak demand — but this remains impossible without a proper digital solution

3. What We Are Building

We are building the Seasonal Production Planning Calendar for Sharadha Stores — a working prototype that solves this problem end to end.
How the system works:

- Step 1 – User logs into the Seasonal Production Planning Calendar system and sees the main dashboard with current status at a glance
- Step 2 – Staff enters data through the Seasonal Production Planning Entry Form with all fields validated before saving
- Step 3 – The capacity and utilisation tracking engine processes entered data and applies business rules specific to this system
- Step 4 – The Seasonal Production Planning Dashboard shows all records with live status, filters, and sortable columns
- Step 5 – Automated alerts fire when thresholds or deadlines are reached as defined in the system
- Step 6 – Management views analytics and reports showing trends, summaries, and actionable insights
- Step 7 – Admin configures system rules, reviews all records, and generates export reports
- Step 8 – The fully tested deployed system is accessible through a public URL with all features working

4. Technology Stack

The following technologies will be used in this prototype:

Layer	Technology	Role in This Project
Frontend	HTML, CSS, JavaScript / React	Seasonal Production Planning Entry Form, Seasonal Production Planning Dashboard, Detail & History View, analytics charts, detail views
Backend	Node.js Express or Python Flask	seasonal_production_planning APIs, capacity and utilisation tracking engine, report generation, data aggregation
Database	MySQL / PostgreSQL	seasonal_production_planning, orders, inventory tables with relationships, indexes, and audit logs
Charts	Chart.js / Recharts	Performance trend charts, comparison analytics, status distribution visualisations

5. Team Structure

Each team has exactly 3 students. The work is divided by role throughout the entire 26-working-day internship:

Student 1 Frontend	Student 2 Backend	Student 3 Testing & Deployment
Seasonal Production Planning Entry Form, Seasonal Production Planning Dashboard, Detail & History View	seasonal_production_planning APIs, capacity and utilisation tracking engine, reports, data validation	Manual testing, Postman API testing, calculation accuracy testing, GitHub, deployment validation

6. What Students Will Build by End of 26 Working Days

- A complete Seasonal Production Planning Calendar system for Sharadha Stores
- Seasonal Production Planning Entry Form with data entry, validation, and database storage
- Seasonal Production Planning Dashboard showing live records with filters and search
- capacity and utilisation tracking engine with accurate, tested business logic
- Automated alerts and notifications for time-sensitive items
- Analytics and reports with trend charts and data export
- A deployed prototype accessible at a live public URL
- Full documentation, PPT, GitHub repository, and demo video

7. Review Milestones

There are 3 mandatory review presentations. Missing any review without valid reason = FAIL in internship evaluation.

Review	Date	What to Present
Review1	06 June 2026	Company intro, project title, problem statement, objectives, abstract
Review2	19-20 June 2026	Literature survey, system analysis, architecture, DB design, initial code
Review3	29-30 June 2026	Full working prototype demo, report, PPT, GitHub, demo video

Day-wise Plan

Each working day is divided into 3 role-specific task columns. Student 1 = Frontend, Student 2 = Backend, Student 3 = Testing & Deployment. All three work in parallel.

Day1	Week1	01 June 2026 (Mon) - Introduction Session

The instructor explains Sharadha Stores's background, the Seasonal Production Planning Calendar project overview, the exact problem it solves, the technology stack, team roles, and the complete 26-day plan. No coding today — every student must fully understand what "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity" means in practice and why "Prevents the most common failure mode of homemade food businesses — running out of stock during peak demand".

Tasks - All 3 Students

Student 1 - Frontend

- Read the Introduction Section twice and summarise it in your own words
- Understand all frontend screens: Seasonal Production Planning Entry Form, Seasonal Production Planning Dashboard, Detail & History View, Admin Configuration Panel, Reports & Analytics Dashboard
- Sketch the Seasonal Production Planning Entry Form on paper — write every field name you expect based on: "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity"
- Sketch the Seasonal Production Planning Dashboard — what columns, filters, and actions should it show?
- Start Day 1 logbook entry explaining your understanding of the business problem

Student 2 - Backend

- Read the Introduction Section twice
- Understand the backend data model — the system must store and process: Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity
- List every data entity needed: seasonal_production_planning, orders, inventory
- Write the 5 specific business rules this system must enforce based on "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity"
- Start Day 1 logbook entry

Student 3 - Testing & Deployment

- Read the Introduction Section twice
- Understand why testing this system matters — specifically what happens if admin plans production batches for upcoming festivals by map has errors
- Research: what is a test case, what is a test tracker, what is regression testing
- Create GitHub account if not already done
- Start Day 1 logbook entry

Deliverable	All students understand exactly what Seasonal Production Planning Calendar does and why Sharadha Stores needs it. Day 1 logbook entries written.

Day2	Week1	02 June 2026 (Tue) - Problem Statement + Abstract

Students write the problem statement and project abstract specific to Seasonal Production Planning Calendar at Sharadha Stores.

These are the primary Review 1 deliverables and must reference the actual business problem, not generic software descriptions.

Tasks - All 3 Students

Student 1 - Frontend

- Write 4-5 sentences describing exactly what goes wrong when "admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity" is done manually
- List all screens the frontend needs with brief purpose: 1. Seasonal Production Planning Entry Form, 2. Seasonal Production Planning Dashboard, 3. Detail & History View, 4. Admin Configuration Panel, 5. Reports & Analytics Dashboard
- Write the abstract: what Seasonal Production Planning Calendar does, who uses it, what changes for Sharadha Stores after it is built
- Push problem statement and abstract to /docs in GitHub

Student 2 - Backend

- Write the problem statement from the data and business logic perspective
- Identify the 3 most critical business rules: what must the system validate, calculate, or enforce for Seasonal Production Planning Calendar
- Write the technical abstract explaining API design, database structure, and core processing logic
- Push to /docs in GitHub

Student 3 - Testing & Deployment

- Create GitHub repository named after the project
- Set up /frontend, /backend, /docs, /tests folders
- Push README with title "Seasonal Production Planning Calendar", company "Sharadha Stores", team names, tech stack
- Push both problem statement drafts to /docs

Deliverable	Problem Statement and Abstract specific to Seasonal Production Planning Calendar completed and pushed. GitHub repository set up.

Day 3	Week 1	03 June 2026 (Wed) - Project Objectives + Dev Setup

Each student writes 5 project objectives and sets up their development environment. Every student must have a working local setup by end of day.

Tasks - All 3 Students

Student 1 - Frontend

- Write 5 objectives from the frontend user perspective — each testable and specific to Seasonal Production Planning Calendar
- Install VS Code, Node.js, create React or HTML/CSS/JS project folder
- Create placeholder files for: SeasonalProductionPlanningEntryForm.jsx, SeasonalProductionPlanningDashboard.jsx, Detail&HistoryView.jsx, AdminConfigurationPanel.jsx
- Verify app runs in browser without errors, push to GitHub

Student 2 - Backend

- Write 5 objectives from the backend and data perspective — reference capacity and utilisation tracking engine in at least one
- Install Node.js Express or Python Flask
- Create server with GET /health returning {"status":"ok","project":"seasonal-production-planning-c"}
- Verify server runs on localhost, push to GitHub

Student 3 - Testing & Deployment

- Write 5 objectives from the testing perspective — include one about accuracy of capacity and utilisation tracking engine
- Install Postman, test GET /health route
- Document first test: what tested, expected, actual result
- Push first test result to /tests in GitHub

Deliverable	5 objectives per student. Frontend running. Backend health route working. First test documented.

Day 4	Week 1	04 June 2026 (Thu) - Use Case Design + Wireframes

All use cases and wireframes designed before any feature coding. This is the planning foundation for Seasonal Production Planning Calendar.

Tasks - All 3 Students

Student 1 - Frontend

- Draw detailed wireframes for all 5 screens: Seasonal Production Planning Entry Form, Seasonal Production Planning Dashboard, Detail & History View, Admin Configuration Panel, Reports & Analytics Dashboard
- For Seasonal Production Planning Entry Form: label every field from "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity" — field names are: Admin, plans, production, batches, upcoming
- For Seasonal Production Planning Dashboard: show columns, filter options, status badges, and action buttons
- Upload all wireframes to /docs in GitHub

Student 2 - Backend

- Draw use case diagram with all actors and their actions for Seasonal Production Planning Calendar
- Write full API list: POST /api/seasonal_production_planning, GET /api/seasonal_production_planning, GET /api/seasonal_production_planning/:id, PUT /api/seasonal_production_planning/:id, plus routes for capacity and utilisation tracking engine
- For each API: define exact request fields (from "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity") and response format
- Write capacity and utilisation tracking engine business rules in pseudocode
- Push use case diagram and API spec to /docs

Student 3 - Testing & Deployment

- Review wireframes — does every item mentioned in "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity" appear somewhere in the screens?
- Write 15 test cases: 3 per screen covering happy path, validation error, and edge case
- Push test cases to /docs in GitHub

Deliverable	All wireframes with labelled fields. Use case diagram. Full API spec. 15 test cases. All pushed.

Day 5	Week 1	05 June 2026 (Fri) - Review 1 Preparation

Build Review 1 PPT and run complete timed mock presentation. Every slide must be finished.

Tasks - All 3 Students

Student 1 - Frontend

- Build PPT: Title, Sharadha Stores background, problem statement, wireframes for all 5 screens
- Each wireframe slide must show field names and explain the screen in one sentence
- Practice frontend section — time it to exactly 3 minutes

Student 2 - Backend

- Build PPT: Proposed solution, tech stack table, architecture sketch, API list, capacity and utilisation tracking engine design
- Prepare plain-language explanation of capacity and utilisation tracking engine for non-technical reviewer
- Practice backend section — 3 minutes

Student 3 - Testing & Deployment

- Build PPT: 5 objectives, abstract, GitHub screenshot, test plan with 15 cases listed
- Verify ALL Day 2-4 documents are in GitHub
- Run full team mock — target 9 minutes, note what needs improvement

Deliverable	Review 1 PPT with all sections completed. Mock presentation done and timed.

Day 6	Week 1	06 June 2026 (Sat) - REVIEW PRESENTATION 1

Team presents company background, Seasonal Production Planning Calendar overview, problem statement, objectives, abstract,

wireframes, tech stack, and GitHub to the reviewer.

Tasks - All 3 Students

Student 1 - Frontend

- Present wireframes — explain each screen using specific field names from "Admin plans production batches for upcoming festivals by mapping expected order volumes to ingredient procurement timelines and production capacity"
- Show the user journey from first login to completing the main task
- Note every piece of feedback the reviewer gives

Student 2 - Backend

- Present problem statement, proposed solution, capacity and utilisation tracking engine design, and tech stack
- Show GitHub repository live — folder structure and document quality
- Note all reviewer feedback on technical decisions

Student 3 - Testing & Deployment

- Present objectives, abstract, and test plan with 15 cases
- After review: compile ALL three students feedback into one team document
- Push feedback document to /docs in GitHub

Deliverable	Review 1 completed. Score received. All reviewer feedback compiled into one document and pushed.

Day 7	**Week 2**	**08 June 2026 (Mon) - Review 1 Feedback + Week 2 Prep**

Apply every piece of Review 1 feedback. Start Week 2 research.

Tasks - All 3 Students

Student 1 - Frontend

- Update all wireframes based on every reviewer comment
- Research 2 existing systems that handle any part of "Admin plans production batches for upcoming festivals by map" — study their UI
- Begin building the Seasonal Production Planning Entry Form in HTML/React with the field names from the wireframe
- Push updated wireframes and initial form code to GitHub

Student 2 - Backend

- Update problem statement and objectives documents based on feedback
- Research how existing backend systems handle capacity and utilisation tracking engine
- Update API spec if reviewer suggested changes
- Push all updated documents to GitHub

Student 3 - Testing & Deployment

- Update all 15 test cases based on reviewer feedback
- Create detailed test tracker: ID, Module, Description, Expected, Actual, Status, Date
- Collect 3 references for the literature survey
- Push all updates to GitHub

Deliverable	All Review 1 feedback applied. Initial form code started. Test tracker created. 3 references collected.

Day 8	**Week 2**	**09 June 2026 (Tue) - Literature Survey + Existing System Analysis**

Research existing systems and document findings. This forms a key Review 2 deliverable and report chapter.

Tasks - All 3 Students

Student 1 - Frontend

- Study 2 systems that handle something similar to "Seasonal Production Planning Calendar" — document their UI design, features, and limitations
- Write a 300-word comparison noting what the proposed system does differently or better
- Push analysis with screenshots/sketches to /docs

Student 2 - Backend

- Study 2-3 systems from a backend/data architecture perspective
- Write Existing System Analysis: System Name, Core Features, Technical Limitations, Gap that Seasonal Production Planning Calendar fills
- State clearly in one paragraph why Seasonal Production Planning Calendar is specifically suited to Sharadha Stores's needs
- Push to /docs

Student 3 - Testing & Deployment

- Summarise each of 3 references in 5 bullets: what it is, key finding, methodology, result, relevance
- Build the Literature Survey document with proper citations
- Verify the survey logically connects to the problem of "Admin plans production batches for upcoming festivals by map"
- Push to /docs

Deliverable	Existing System Analysis completed. Literature Survey with 3 references. All pushed to GitHub.

Day 9	**Week 2**	**10 June 2026 (Wed) - Proposed System + Database Design**

Fully describe the proposed system and design the complete database schema. Nothing gets built without this plan.

Tasks - All 3 Students

Student 1 - Frontend

- Write proposed system description for each of 5 screens: purpose, actors, inputs, outputs
- Create comparison table: Current Process at Sharadha Stores (manual) vs Seasonal Production Planning Calendar (digital)
- Update wireframes to fully align with proposed system description
- Push to /docs

Student 2 - Backend

- Design complete schema: seasonal_production_planning(id, Admin, plans, production, status, created_at, updated_at); orders(id, Admin, plans, production, status, created_at, updated_at); inventory(id, Admin, plans, production, status, created_at, updated_at)
- Create ER diagram with all relationships, foreign keys, and cardinality
- Document all validation rules and the exact capacity and utilisation tracking engine logic with thresholds and edge cases
- Push ER diagram and schema document to /docs

Student 3 - Testing & Deployment

- Review ER diagram — does it cover every field mentioned in the W description?
- Write comprehensive test plan: 28 test cases covering all modules
- Push to /docs

Deliverable	Proposed System Description written. ER Diagram with full schema. Test plan with 28+ cases. All pushed.

Day 10	**Week 2**	**11 June 2026 (Thu) - Database + Seasonal Production Planning Entry Form**

Database created from Day 9 schema. First core API built. Frontend builds the main entry form.

Tasks - All 3 Students

Student 1 - Frontend

- Build complete Seasonal Production Planning Entry Form with all fields: Admin, plans, production, batches, upcoming
- Apply CSS for clean professional layout — suitable for a business tool
- Add client-side validation with specific error message per field
- Test at desktop (1280px) and mobile (375px)

Student 2 - Backend

- Create database and run migration scripts for all tables: seasonal_production_planning, orders, inventory
- Build POST /api/seasonal_production_planning: validate required fields, insert record, return created record with ID
- Build GET /api/seasonal_production_planning: return all records sorted by created_at DESC
- Test POST with valid data, test POST with each required field missing
- Push to GitHub

Student 3 - Testing & Deployment

- Test POST with 5 complete records — verify all 5 appear in database with correct values
- Test POST with 3 different validation errors — verify each returns specific, correct error message
- Test GET — verify all 5 records returned in correct order
- Document all 10 results in test tracker

Deliverable	Database created. POST and GET /api/seasonal_production_planning working. Seasonal Production Planning Entry Form built. All 10 tests documented.

Day 11	**Week 2**	**12 June 2026 (Fri) - Seasonal Production Planning Dashboard + Detail API**

Main dashboard shows real data. Detail retrieval added.

Tasks - All 3 Students

Student 1 - Frontend

- Build Seasonal Production Planning Dashboard as data table with columns: Admin, plans, production, batches, Status, Actions
- Connect to GET /api/seasonal_production_planning — replace dummy data with real API data
- Add loading spinner and empty state message
- Add status filter and search by Admin
- Push to GitHub

Student 2 - Backend

- Build GET /api/seasonal_production_planning/:id returning single complete record
- Build filtering in GET /api/seasonal_production_planning?status=X&search=Y
- Add pagination: 20 per page with total count in response
- Test all routes in Postman, push to GitHub

Student 3 - Testing & Deployment

- Test GET by valid ID — all fields returned
- Test GET by invalid ID — 404 returned
- Test pagination — page 1 vs page 2
- Test filter and search
- Document all 6 results in test tracker

Deliverable	Seasonal Production Planning Dashboard showing live data. GET by ID. Pagination. Filter and search. All tested.

Day 12	**Week 2**	**13 June 2026 (Sat) - GitHub Workflow + Integration Check**

All code merged via pull requests. Complete flow tested end-to-end.

Tasks - All 3 Students

Student 1 - Frontend

- Create feature/frontend branch, push all completed screen files, raise PR
- After merge: verify all screens still work from merged main branch
- Fix any merge conflicts

Student 2 - Backend

- Create feature/backend branch, push backend files + migration scripts, raise PR
- After merge: test all APIs in Postman from merged code
- Ensure migration script is in /backend/migrations/ folder

Student 3 - Testing & Deployment

- Review both PRs: no hardcoded credentials, complex logic has comments, correct file naming
- After merge: test full flow — fill Seasonal Production Planning Entry Form, submit, see it in Seasonal Production Planning Dashboard, click for detail
- Update README with exact steps to run project locally
- Document end-to-end test in tracker

Deliverable	All code merged via reviewed PRs. End-to-end flow tested. README updated. Main branch stable.

Day 13	**Week 3**	**15 June 2026 (Mon) - Core Logic: Capacity and utilisation tracking engine**

The most important logic of Seasonal Production Planning Calendar — the capacity and utilisation tracking engine — is built and thoroughly tested. This is the system's core value.

Tasks - All 3 Students

Student 1 - Frontend

- Build the output/result screen showing what the capacity and utilisation tracking engine produces
- Use clear visual indicators: calculated value with context label and trend indicator
- Build static mockup with hardcoded test data first to verify layout
- Push mockup to GitHub

Student 2 - Backend

- Build the capacity and utilisation tracking engine: query database for relevant records, apply the business rules from "Admin plans production batches for upcoming festivals by mapping expected order ", return structured result
- Manually calculate expected output for 3 known test inputs — verify API matches
- Handle edge cases: null values, zero records, boundary conditions
- Add inline comments explaining every step of the logic
- Push to GitHub

Student 3 - Testing & Deployment

- Write 10 test cases for the core logic:
- — 3 typical cases (manually calculate expected output and verify)
- — 3 edge cases (boundary values, minimum, maximum)
- — 2 error cases (missing data, invalid input)
- — 2 regression cases (fix edge case without breaking typical case)
- Run all 10, document in test tracker

Deliverable	Capacity and utilisation tracking engine built and tested with 10 cases. Output screen built. All results documented.

Day 14	**Week 3**	**16 June 2026 (Tue) - Review 2 Prep + Literature Survey Final**

All Week 2 deliverables finalised. Review 2 PPT built and rehearsed.

Tasks - All 3 Students

Student 1 - Frontend

- Finalise all frontend screens: Seasonal Production Planning Entry Form, Seasonal Production Planning Dashboard, Detail & History View
- Build Review 2 PPT: final wireframes + actual screenshots, UI decisions, user flow diagram
- Run mock of frontend section — 3-4 minutes

Student 2 - Backend

- Finalise all backend code with comments
- Build Review 2 PPT: architecture, ER diagram, capacity and utilisation tracking engine with worked example, API summary
- Run mock of backend section

Student 3 - Testing & Deployment

- Finalise literature survey with 5+ references each with 5-bullet summary
- Build Review 2 PPT: literature survey highlights, existing system analysis, test progress, GitHub
- Run full team mock — target 11 minutes
- Verify GitHub has all deliverables in /docs

Deliverable	Literature survey with 5 references finalised. All Week 2 deliverables complete. Review 2 PPT done. Mock completed.

Day 15	Week 3	17 June 2026 (Wed) - Architecture Diagram + Remaining Plan

Complete architecture documented. Remaining work clearly planned.

Tasks - All 3 Students

Student 1 - Frontend

- Create architecture diagram: Frontend → Backend API → Database
- Label every API call: POST/Admin, POST/plans, POST/production...
- Export PNG, push to /docs
- Plan remaining screens: edit form, orders management, reports

Student 2 - Backend

- Validate architecture diagram from backend perspective
- Plan remaining routes: PUT /api/seasonal_production_planning/:id, PATCH status, GET /api/orders
- Plan the summary/analytics route
- Push remaining work plan to /docs

Student 3 - Testing & Deployment

- Review architecture for testing gaps
- Write integration test plan: 8 end-to-end scenarios covering critical user journeys
- Write 3 specific edge cases for the capacity and utilisation tracking engine with exact input values
- Push integration test plan to /docs

Deliverable	Architecture diagram pushed. Remaining work planned. Integration test plan with 8 scenarios documented.

Day 16	Week 3	18 June 2026 (Thu) - CRUD Complete + Frontend Integration

Full CRUD cycle completed. Frontend connected end-to-end for all operations.

Tasks - All 3 Students

Student 1 - Frontend

- Build edit form: clicking a record in Seasonal Production Planning Dashboard opens form pre-filled with current values
- Connect edit form to PUT API with success/error feedback
- Add status change buttons with confirmation dialogs
- Add filter tabs on Seasonal Production Planning Dashboard: All, Active, Completed, Archived
- Test full create → edit → status change flow in browser

Student 2 - Backend

- Build PUT /api/seasonal_production_planning/:id with same validation as POST
- Build PATCH /api/seasonal_production_planning/:id/status for status-only changes

- Build GET /api/orders and POST /api/orders
- Test all new routes in Postman, push to GitHub

Student 3 - Testing & Deployment

- Test full CRUD: create → verify in dashboard → edit → verify change saved → deactivate → verify removed from active view
- Test status transitions: verify only valid transitions are accepted
- Test PUT with non-existent ID — 404 returned
- Document all 8 CRUD tests in test tracker

Deliverable	Full CRUD completed. Frontend connected for all operations. All 8 CRUD tests documented.

Day 17	**Week 3**	**19 June 2026 (Fri) - REVIEW PRESENTATION 2 — Day 1**
First batch presents literature survey, existing system analysis, architecture, DB design, and working code for Seasonal Production Planning Calendar.		

Tasks - All 3 Students

Student 1 - Frontend

- Present all screens with live demo: show Seasonal Production Planning Entry Form submitting real data and Seasonal Production Planning Dashboard displaying it
- Answer questions about UI design, field choices, and user experience
- Note all reviewer feedback

Student 2 - Backend

- Present architecture, ER diagram, and capacity and utilisation tracking engine with a worked example calculation
- Show POST, GET, and PUT APIs working in Postman with real data
- Note all feedback

Student 3 - Testing & Deployment

- Present literature survey, existing system analysis, test results, GitHub
- Show test tracker with current pass rate
- Compile all team feedback after review and push

Deliverable	Review 2 Day 1 completed. Score received. All feedback compiled and pushed.

Day 18	**Week 3**	**20 June 2026 (Sat) - REVIEW PRESENTATION 2 — Day 2 + Reports**
Remaining teams present Review 2. All teams build the reports and analytics module.		

Tasks - All 3 Students

Student 1 - Frontend

- Present Review 2 if in second batch
- Build Reports screen: summary counts, performance trend charts, date-range filter, CSV export
- Add bar chart and line chart with real data from API
- Push to GitHub

Student 2 - Backend

- Present Review 2 if in second batch
- Build GET /api/reports/summary: counts by status, totals, and 30-day time series for charts
- Apply all Review 2 feedback to backend
- Push to GitHub

Student 3 - Testing & Deployment

- Present Review 2 if in second batch

- Test reports with 3 date ranges — verify counts are correct
- Test chart renders with 0, 1, and 50+ records
- Document 5 report test cases

Deliverable	Review 2 Day 2 completed. Reports screen with charts built. All pushed.

Day 19	**Week 4**	**22 June 2026 (Mon) - Full Integration + Alerts/Notifications**
All components integrated into a complete working flow with automated alerts.		

Tasks - All 3 Students

Student 1 - Frontend

- Connect all screens via navigation: Seasonal Production Planning Entry Form → Seasonal Production Planning Dashboard → Detail & History View → Admin Configuration Panel → Reports & Analytics Dashboard
- Add breadcrumb navigation
- Build home dashboard summary widget with key metric counts
- Test complete navigation in browser

Student 2 - Backend

- Build GET /api/dashboard/summary returning key metrics for dashboard widget
- Connect all remaining frontend-to-API links
- Test full data pipeline with 10 realistic records
- Fix any integration bugs, push to GitHub

Student 3 - Testing & Deployment

- Run all 8 integration scenarios from Day 15 test plan
- Document exact steps, data used, and outcome for each
- Test 3 specific edge cases from Day 15 with exact input values
- Create bug report for every failure found
- Push all results to test tracker

Deliverable	All navigation connected. Alerts/summary widget working. 8 integration scenarios tested. Bugs documented.

Day 20	**Week 4**	**23 June 2026 (Tue) - Detail Page + 15-Record Test**
Detail page built. Full integration test with 15 realistic records validates entire system.		

Tasks - All 3 Students

Student 1 - Frontend

- Build detail page: click a record in Seasonal Production Planning Dashboard → see ALL information including every field, related data, and action history
- Add print/export button on detail page
- Show linked data from orders and inventory if applicable
- Test detail page for 5 different records

Student 2 - Backend

- Build GET /api/seasonal_production_planning/:id/detail returning record with all related data joined
- Test join for records with related data (exists) and without (empty arrays returned cleanly)
- Push to GitHub

Student 3 - Testing & Deployment

- Insert 15 realistic test records covering the full variety of statuses and values
- For each of 15 records: verify dashboard shows correctly, open detail page, verify all fields accurate, perform one action, verify change persists
- Check data consistency: same record shows same values on dashboard, detail page, and in reports
- Document all findings — any inconsistency is a bug

- Push to test tracker

Deliverable	Detail page built. 15 realistic records tested across all screens. Data consistency verified. Bugs documented.

Day 21	Week 4	24 June 2026 (Wed) - UI Polish + Error Handling
System polished visually. All errors handled. All edge cases tested.		

Tasks - All 3 Students

Student 1 - Frontend

- Visual audit: one font family, consistent button sizes, 8px spacing grid, consistent colour use
- Add loading spinners for every API call
- Add empty state messages for every list screen
- Test at 375px, 768px, 1280px and fix all responsive issues
- Push all CSS fixes

Student 2 - Backend

- Add try-catch to EVERY route — no unhandled promise rejections allowed
- Standardise errors: {"success":false,"message":"specific message","code":400/404/500}
- Add input sanitisation: strip HTML and special characters from text fields
- Test server stability: send malformed JSON, empty body, extra-large payload
- Push final backend

Student 3 - Testing & Deployment

- Test the 3 edge cases from Day 15 in the fully integrated system
- Test all empty states
- Intentionally trigger every error and verify the message is user-friendly
- Test from a clean empty database — does everything work from scratch?
- Calculate test pass rate. Document all results.

Deliverable	Visual audit complete. Error handling on every route. Edge cases tested. Mobile verified. Deployment-ready.

Day 22	Week 4	25 June 2026 (Thu) - Deployment
System deployed to a public URL accessible without any local environment.		

Tasks - All 3 Students

Student 1 - Frontend

- Run production build, fix any build errors
- Update all API URLs to deployed backend URL
- Deploy frontend to Vercel (connect GitHub repo)
- Verify every screen loads at deployed URL
- Screenshot every screen for the project report

Student 2 - Backend

- Deploy backend to Render or Railway
- Set environment variables: DATABASE_URL, PORT, any secrets — never in code
- Deploy PostgreSQL to Supabase or Railway
- Run all migration scripts on production database
- Test EVERY API endpoint at deployed URL in Postman

Student 3 - Testing & Deployment

- Test complete end-to-end on deployed URL: create record → dashboard → detail → edit → verify
- Test on real mobile device
- Add live URL to README, push

- Write deployment guide in /docs with all environment variables listed

Deliverable	Backend deployed. Database deployed with all tables. Frontend deployed and connected. All features verified at live URL.

Day 23	Week 4	26 June 2026 (Fri) – Final Testing + Report Part 1

Comprehensive final testing on deployed system. Bugs fixed. First half of project report written.

Tasks – All 3 Students

Student 1 – Frontend

- Test every feature on deployed URL at mobile, tablet, desktop
- Fix all responsive issues on live URL
- Write Report Chapter 1 (Introduction) and Chapter 4 (UI Design)

Student 2 – Backend

- Verify capacity and utilisation tracking engine produces correct results on deployed backend with 5 specific test inputs
- Fix any production bugs
- Write Report Chapter 3 (System Design): architecture, DB schema, API design, core logic

Student 3 – Testing & Deployment

- Execute complete final test suite on deployed URL — run every test case in test tracker
- Mark each Pass or Fail with actual result documented
- Create Final Bug Report: ID, Module, Description, Steps to Reproduce, Severity, Status
- Write Report Chapter 5 (Testing): strategy, cases, results table, bug report, pass rate

Deliverable	Final test suite executed on deployed URL. Bug report created. Chapters 1, 3, 4, 5 written.

Day 24	Week 4	27 June 2026 (Sat) – Report Part 2 + Demo Video + PPT

Project report completed. Demo video recorded. Final PPT built and rehearsed.

Tasks – All 3 Students

Student 1 – Frontend

- Record 4-5 minute demo: Sharadha Stores's problem → Seasonal Production Planning Entry Form in use → Seasonal Production Planning Dashboard with multiple records → capacity and utilisation tracking engine output → reports → detail page → mobile view
- Upload to Google Drive (shareable) + YouTube (unlisted), add links to README
- Build Final PPT Part 1: Title, Problem, Solution Screenshots of all 5 screens, live URL

Student 2 – Backend

- Write Report Chapter 2 (Literature Survey) and Chapter 6 (Conclusion and Future Work)
- Write References in IEEE format
- Proof-read full report for technical accuracy

Student 3 – Testing & Deployment

- Proof-read report for grammar and formatting consistency
- Export as PDF, push to /docs
- Build Final PPT Part 2: Testing results table, Deployment screenshot, Conclusion, 3 future enhancements
- Run full team mock — 13 minutes target

Deliverable	Complete report as PDF. Demo video with live system recorded. Final PPT with all sections. Mock done.

Day 25	Week 5	29 June 2026 (Mon) – REVIEW 3 — Day 1 + Submissions

First batch presents Review 3 with live system demo and submits all deliverables.

Tasks - All 3 Students

Student 1 - Frontend

- Open deployed URL live — demo: Seasonal Production Planning Entry Form → Seasonal Production Planning Dashboard → detail → capacity and utilisation tracking engine output → reports → mobile
- Answer all questions about frontend implementation
- Note final feedback

Student 2 - Backend

- Present architecture, capacity and utilisation tracking engine with live worked example, database design
- Show 3 API routes live in Postman with real deployed data
- Answer all technical questions

Student 3 - Testing & Deployment

- Present test results: pass rate, bug report, deployment confirmation
- Submit all deliverables: GitHub URL, Deployed URL, Report PDF, PPT, Demo Video, Project Diary
- Confirm reviewer received everything

Deliverable	Review 3 Day 1 completed. Score received. All deliverables submitted and confirmed.

Day 26	Week 5	30 June 2026 (Tue) - REVIEW 3 — Day 2 + Internship Close

Remaining teams present Review 3. Internship closes with individual reflections.

Tasks - All 3 Students

Student 1 - Frontend

- Present Review 3 if in second batch
- Share 3 specific frontend skills you developed in 26 days
- Write final logbook entry and submit to instructor

Student 2 - Backend

- Present Review 3 if in second batch
- Share 3 specific backend/logic learnings from building Seasonal Production Planning Calendar
- Write final logbook entry and submit

Student 3 - Testing & Deployment

- Present Review 3 if in second batch
- Share 3 specific testing learnings from this project
- Write final logbook, confirm all submissions complete, officially close internship

Deliverable	All Review 3 presentations done. All deliverables confirmed. Logbooks submitted. Internship officially closed.